

Feature: Treatment category pages

From build-plan: feature 4 **Status:** verified **Branch:** `feature/treatment-category-pages`

Goal

Five pages, one template, at `/treatments/[category]` : Facials, Skin Treatments, Body, Massage, Laser and IPL. Every treatment in the category listed with its description, price and duration, grouped where the data groups it, filterable by concern, each row leading to a way to book.

These are the pages meant to rank for the category terms. A treatment without its own detail page is an anchor here and nowhere else, so this page has to be complete rather than a menu pointing elsewhere.

The header shipped in `cacc0e0` builds its dropdowns from the data: five category links, forty treatment links, eight concern links. Every one of them 404s today. This feature turns the five category links live. The forty and the eight stay broken until features 5 and 6.

Design reference

`prototypes/facials.html` . Page head, then one `.group` block per facial group with its own `h2` heading, then the rows. Tokens are already in `globals.css` .

Two things in that prototype stay out. The `.addons` block has no data behind it (see Open questions), and the closing "Let us look first" band is the consultation CTA, which is in scope.

In scope

- **Route** `app/treatments/[category]/page.tsx` , static per slug via `generateStaticParams` over `CATEGORIES` , matching the project's rendering rule. An unknown segment calls `notFound()` .
- **Metadata** per category: title from the label, description from new per-category copy. Both real strings, not generated filler.
- **Hero**: breadcrumb (Home / label; there is no `/treatments` index to link), eyebrow, `h1` of the category label, intro copy, and the existing `cat*` photo slot for that category.
- **Service rows**, reading `getServicesByCategory` : name, description, duration, and price rendered three ways - `$X` , `from $X` when `priceFrom` , and "At consultation" when `price` is null.
- **Grouping**: when a category has `groups` , render one block per group with its `heading` as an `h2` , services placed by their `group` field. Facials is the only category with groups today; the other four render one flat list. A service whose `group` matches nothing falls into an ungrouped block rather than disappearing.

- **Concern filter:** chips above the list, one per concern that has at least one service *in this category*. Selecting one shows only matching rows, selecting it again clears it.
- **Eminence tag** on the four services carrying `eminence: true`, using the existing `Tag` component.
- **Row action:** priced services get "Book", unpriced get "Ask about pricing". Both link to `/contact?treatment=<slug>`, which feature 9 already reads and prefills. Real booking is feature 11.
- **Closing consultation band**, same pattern as the contact page's.
- **Data:** add `intro` and `metaDescription` to `Category` in `types/services.ts` and fill both for all five in `data/categories.ts`, beside the existing blurbs and group headings that also await her sign-off.

Out of scope

- Treatment detail pages (feature 5) and concern landing pages (feature 6). The forty and eight header links stay 404 after this.
- Real booking (feature 11). Rows point at `/contact`.
- JSON-LD and schema (feature 10).
- The prototype's add-ons block (Open questions).
- Any change to `/contact`, the home page, or the header.

Build loop

`workflow.stepReview` is `feature`, so review comes at the end of the feature rather than per step. `workflow.checkpointCommits` is `disabled`, so no commits during the build. `/complete` makes the one commit.

Build steps

- ☒ **1. Route, params and hero.** Add the `Category` fields, fill them for all five, and build the page shell. *Done when:* `npm run build` lists five new static routes under `/treatments/`, `/treatments/facials` renders the breadcrumb, `h1` "Facials", its intro and its photo, and an unknown segment such as `/treatments/nonsense` returns 404.
- ☒ **2. Service rows and grouping.** Render every service in the category. *Done when:* the five pages together render all 40 services, per-page counts match `getServicesByCategory`, facials shows its four group headings in `CATEGORIES` order, and the three price forms all appear (a fixed price, a `from` price, and "At consultation" on the three unpriced services). The two `variantOf` messages render as their own rows, since each carries its own price and duration.
- ☒ **3. Concern filter.** A client shell owning only the selected concern. *Done when:* chips render only for concerns with at least one service in that category, selecting one hides non-matching rows, selecting it

again restores them, chips carry `aria-pressed`, and the built client chunk for the route does **not** contain service descriptions. *Caveat on the last clause:* `ConcernFilter`'s own chunk is 1,389 bytes and carries no service data, so this feature adds none. The route still loads a chunk that does, because the header nav built in `cacc0e0` pulls `data/services.ts` into the client graph on every page including the home page. That is a pre-existing defect in the header, not this feature, and repairing it here would be out of scope. Raised for `/audit`.

- ☒ **4. Row actions and tags.** Book and ask links, Eminence tags. *Done when:* a priced row links to `/contact?treatment=<slug>` labelled "Book", an unpriced row is labelled "Ask about pricing", following one lands on `/contact` with the topic and message prefilled, and the four Eminence services show the tag.
- ☒ **5. Consultation band and header check.** Close the page and confirm the nav. *Done when:* each of the five pages ends with the consultation CTA, all five header category links resolve with no 404, and the pages have no horizontal overflow at 375px.

Files / areas

| Path | Change | |---|---| | `app/treatments/[category]/page.tsx` | new, the template | | `components/treatments/ServiceList.tsx` | new, server, grouping | | `components/treatments/ServiceCard.tsx` | new, server | | `components/treatments/ConcernFilter.tsx` | new, client shell plus the server-rendered style | | `types/services.ts` | add `intro`, `metaDescription` to `Category` | | `data/categories.ts` | fill both for all five | | `lib/services.ts` | add a helper for concerns present in a category | | `app/globals.css` | untouched in the end, see Data / contracts |

Data / contracts

- **URL:** `/treatments/<segment>`, segment from `Category.segment` (`facials`, `skin`, `body`, `massage`, `laser`). Already produced by `categoryHref`; do not invent a second shape.
- **Price rendering:** `price === null` renders "At consultation" and never "\$0". `priceFrom` renders "from \$X". Whole dollars, no cents.
- **The filter carries no service data to the client.** The server renders every row with `data-concerns="slug slug"`, the client shell holds only the selected slug and sets `data-concern` on the wrapper, and a generated stylesheet hides rows that do not match. The shell receives only `{slug, label, count}[]` for the chips. **Built differently from the first draft:** the rules are emitted by `ConcernFilterStyles` next to the list rather than written into `globals.css`. CSS cannot compare two attribute values, so a static stylesheet would have needed a hand-maintained rule pair per concern that drifts whenever `ConcernSlug` changes. Emitting only the concerns present in that category stays correct by construction, and the same rules also collapse a group whose every card is filtered out. This is the project's stated pattern for partly-interactive sections, and **F-03** in the findings ledger is an open finding about the contact form doing the opposite. Do not repeat it here.

- **Chips only for concerns with matches**, so a filter can never produce an empty list and no empty state is needed.

Testing

No test command is declared in `AGENTS.md`, so there is no runner to add unit tests to and this feature does not add one. Evidence is `npm run build` plus the browser at desktop and 375px, per the done-whens above.

The one machine-checkable claim is step 3's bundle check: grep the built client chunk for a service description string and confirm it is absent.

Notes for the AI

- Server components throughout except `ConcernFilter`, which needs state.
- Reuse `Section`, `Container`, `Eyebrow`, `Lede`, `Photo`, `Tag`, `Divider`. Do not add new layout primitives.
- No new dependencies. The filter is CSS plus one `useState`.
- `bookingId` stays untouched. No booking provider appears anywhere.
- The overview says "45 services"; the data has 40. The data is right. Do not edit the overview from this feature.

Open questions

- **The prototype's add-ons block.** `prototypes/facials.html` shows an add-ons panel with its own short list and prices. Nothing in `SERVICES` models an add-on, and inventing one would be new product data rather than a build decision. Scoped out for now. If she wants it, it is a small data change plus a block, and it can be a fix or a plan item later.

Implementation walkthrough

Category data (`types/services.ts`, `data/categories.ts`) - `Category` gained `intro` and `metaDescription`, filled for all five beside the blurbs and group headings that already awaited her sign-off. Both are real copy drawn from what the data actually supports: the laser intro says the price comes after a look because both laser services genuinely carry `price: null`, and the skin intro hedges because one of its four does. `getCategoryBySegment` sits next to `categoryHref` so the URL shape has exactly one definition in both directions.

The page (`app/treatments/[category]/page.tsx`) - one template, `generateStaticParams` over `CATEGORIES`, and `dynamicParams = false` so an unknown segment is a build-time-guaranteed 404 rather than a runtime branch. `params` is awaited, as Next 15 onward requires. The breadcrumb is Home

then the category, with no `/treatments` link, because no index route exists; inventing one would have been a dead link. The hero reuses the contact page's shape so the two inner pages read as siblings.

Grouping (`ServiceList.tsx`) - facials is the only category with `groups` today, so the component branches: grouped categories render a section per group in `CATEGORIES` order, ungrouped ones render one grid. Services are placed into groups by their `group` field and anything left over lands in a trailing "More treatments" block. That block is empty today (the four facial groups account for all 22) and exists so a service can never silently vanish by having a `group` value nobody defined.

Cards (`ServiceCard.tsx`) - price renders three ways and never as `$0` : "At consultation" for `price: null` , a smaller "from" prefix when `priceFrom` , otherwise the plain figure. The action follows the price rather than being a fixed label, so unpriced treatments say "Ask about pricing" and link to the same `/contact?treatment=<slug>` that feature 9 already reads. `featured` cards span the grid, which matters most on the laser page where both treatments are featured.

The concern filter (`ConcernFilter.tsx`) - the part worth explaining. The obvious build is a client component holding the services and filtering them, which is exactly what put the whole catalogue in the browser on `/contact` (F-03). Instead the server renders every card with `data-concerns="acne pigmentation"` , the client component holds one string and sets `data-concern` on the wrapper, and CSS does the hiding. The client bundle for it is 1,389 bytes and contains no service data.

CSS cannot compare two attribute values, so the rules have to name each concern. Writing them into `globals.css` would have meant a hand-maintained pair per concern that rots the moment `ConcernSlug` changes, so `ConcernFilterStyles` emits them server-side for only the concerns present on that page. The second rule in each pair uses `:has()` to collapse a group whose every card is filtered out, which is what the prototype's `.group.hide` did by hand.

Chips render only for concerns with at least one match in that category, which removes the empty state entirely rather than designing one. The counter is driven by counts computed on the server and passed with the chips, so it stays accurate without the client knowing what it is counting.

Verified - `npx tsc --noEmit` , `npm run lint` , `npm run build` clean, five SSG routes. Against the production server: 22 / 4 / 4 / 8 / 2 cards matching `getServicesByCategory` , the four facial group headings in order, three unpriced treatments showing "At consultation". Filtering facials by Acne went 22 cards to 8 with every visible card genuinely matching and the groups collapsing 4 to 2; `aria-pressed` flipped, the live counter read "8 treatments", and toggling off restored 22. Chips per page were 5 / 4 / 2 / 2 / 4, exactly the concerns present in each. `/treatments/nonsense` returned 404. "Ask about pricing" on a laser treatment landed on `/contact` with the topic preselected and the message prefilled. All five header category links returned 200. At 375px `scrollWidth` equalled `clientWidth` ; the only elements past the edge are the filter chips inside their own horizontal scroller, as the prototype intends.

Known and deliberately not fixed here - the route still loads a client chunk containing service descriptions, because the header nav built in `cacc0e0` pulls `data/services.ts` into the client graph on every page, the home page included. This feature adds nothing to that. F-03 in the ledger attributes it to the contact form, which is not the main cause; the header is. Repairing the nav was out of scope for this feature.